KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 5 Laboratory Sheet

Tractable Structure and Symmetry: Trees, Treewidth, and Breaking Symmetry in Combinatorial Design

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours

Step 9 steps · 5 checkpoints

Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), the course's shared Python 3.12 virtual environment from Week 1 (already carrying `ortools`), a text editor, and paper for two hand-worked tasks (Steps 2 and 5)

1 2 3 ✓ CP1 4 5 ✓ CP2 6 ✓ CP3 7 8 ✓ CP4 9 ✓ CP5

Learning phases: 1 · Pre-instruction | 2 · Content | 3 · Participation | 4 · Assessment | 5 · Follow-through

Why It Matters 1 · Pre-instruction

Every search tree you built in Weeks 3 and 4 could, in the worst case, force you to backtrack. This lab shows you two completely different reasons a problem's own shape can save you that cost before a single branch is explored: a constraint graph that happens to be a tree needs no backtracking at all, and a graph full of interchangeable variables often hides a search space many times larger than the number of genuinely different solutions it actually contains. By the end of the next two hours you will have run both effects on real CP-SAT models, hand-traced the graph arithmetic that predicts them, and added one more symmetry-breaking constraint entirely on your own.

Prior Knowledge 1 · Pre-instruction

This sheet exercises all four units of this week's chapter: tree-structured CSPs and directional arc consistency (Unit 1), cutset conditioning, tree decomposition and treewidth (Unit 2), what makes a CSP tractable versus NP-hard (Unit 3), and symmetry and symmetry-breaking constraints (Unit 4). Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: Confirming the Shared Environment

Step 1: **Confirm this week needs no new package**

What will you do? With the course's shared virtual environment `kod_ornekleri/.venv` active, you will print the installed `ortools` version to confirm the same package Week 1 installed is still there — this week is CP-SAT only (no MiniZinc, no Gurobi), so no new package install is needed.
Why? Every model in this sheet is checked against a real, captured solver run; confirming the exact pinned version before you start is what makes every `ciktikutusu` box below reproducible on your own machine, exactly as it was in Weeks 1–4.

Terminal

```
$ cd kod_ornekleri
$ source .venv/bin/activate
$ python -c "import ortools; print(ortools.__version__)"
```

Expected result: The command prints `9.14.6206`, the exact version Week 1 pinned for the whole course.

Expected Output

```
9.14.6206
```

Tip

If this command raises `ModuleNotFoundError`, your virtual environment is not active in this terminal — re-run `source .venv/bin/activate` from inside `kod_ornekleri` before continuing.

macOS / Linux

`source .venv/bin/activate` adds `(.venv)` to your shell prompt; every `python` command in the rest of this sheet then resolves to this environment's interpreter automatically.

Windows

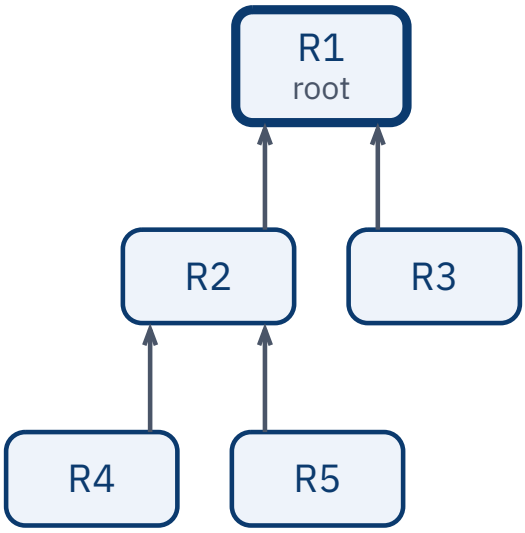
In PowerShell, activate with `.venv\Scripts\Activate.ps1` instead.

2 Unit 1 — Tree-Structured CSPs and Directional Arc Consistency

(See Course Notes Week 5, Unit 1 — Tree-Structured CSPs and Directional Arc Consistency.)

Step 2: **Run tree_csp.py and confirm the zero-backtrack assignment**

What will you do? You will run `tree_csp.py`, which sweeps directional arc consistency (DAC) over a genuine tree-shaped 5-region map (R1 the root, R2 and R3 its children, R4 and R5 the children of R2 — deliberately unlike Week 1's cyclic Australia map), assigns the regions forward, and cross-checks the result against a plain CP-SAT solve of the same instance.
Why? This is Unit 1's central claim, running as real code for the first time: Freuder's tree theorem promises zero backtracks whenever DAC has already been applied along a tree-consistent ordering, and this run is your first direct evidence that the promise holds, not just an argument on paper.



A tree-structured constraint graph — contrast with Week 1's Australia map, which has a triangle (WA–NT–SA) and is not a tree.

The tree-consistent ordering and the reverse-order DAC sweep:

```python
REGIONS = ["R1", "R2", "R3", "R4", "R5"]
EDGES = [("R1", "R2"), ("R1", "R3"), ("R2", "R4"), ("R2", "R5")]

def build_domains() -> dict[str, list[int]]:
    """One 3-colour domain (0, 1, 2) per region, before any DAC filtering."""
    return {region: [0, 1, 2] for region in REGIONS}
```

```python
def apply_dac(domains: dict[str, list[int]]) -> None:
    """Directional arc consistency, processed in reverse tree-consistent
    order (last region to second): for each region, remove from its
    parent's domain every value with no compatible value left in the
    region's own (current) domain.
    """
    for child in reversed(ORDER[1:]):
        parent = PARENT[child]
        domains[parent] = [
            value
            for value in domains[parent]
            if any(is_compatible(value, other) for other in domains[child])
        ]
        print(f"Revise {parent} against {child}: domains[{parent}] = {domains[parent]}")
```

Terminal

```
$ cd hafta05
$ python modeller/tree_csp.py
```

Expected result: Four DAC revisions (R2 against R5, R2 against R4, R1 against R3, R1 against R2, in that reverse-order sweep), then a forward assignment R1 through R5 with zero backtracks, followed by a CP-SAT cross-check confirming the same instance's validity.

Expected Output

```
Initial domains: {'R1': [0, 1, 2], 'R2': [0, 1, 2], 'R3': [0, 1, 2],
'R4': [0, 1, 2], 'R5': [0, 1, 2]}
Revise R2 against R5: domains[R2] = [0, 1, 2]
Revise R2 against R4: domains[R2] = [0, 1, 2]
Revise R1 against R3: domains[R1] = [0, 1, 2]
Revise R1 against R2: domains[R1] = [0, 1, 2]
Assign R1 = Red
Assign R2 = Green
Assign R3 = Green
Assign R4 = Red
Assign R5 = Red
Backtracks needed: 0
CP-SAT cross-check status: OPTIMAL, valid coloring: True
```

Tip

If your terminal is still at the course root, `cd kod_ornekleri` first, then `cd hafta05` — every remaining command in this sheet is run from inside `kod_ornekleri/hafta05`.

Step 3: **Explain, by hand, why no domain shrank**

What will you do? Look again at the trace above: every one of the four DAC revisions leaves `domains[R2]` and `domains[R1]` at all three colours, `[0, 1, 2]`. On paper, explain why this is exactly what Freuder's tree theorem predicts here, and why the forward assignment is still guaranteed backtrack-free even though DAC filtered nothing away.

Why? It is tempting to read "DAC removed nothing" as "DAC did nothing useful." Working out why that reading is wrong is what turns Unit 1's theorem from a fact you accepted on the star-graph practice task into a fact you can defend on an instance with different numbers.

Expected result: Each region has at most two neighbours in this tree (R1 has two children, R2 has one parent and two children, the three leaves have one parent each), and every domain has three colours available. A DAC revision only removes a parent's colour when *every* value in the child's current domain conflicts with it; with three colours and only one value forbidden per neighbour at a time, a parent facing a single child always keeps at least two compatible colours, so no revision here can ever empty out even one value. Freuder's theorem does not require DAC to shrink anything — it only requires DAC to have been applied along a tree-consistent order before the forward pass; the guarantee comes from the tree's shape (one parent per node), not from how much filtering happens to occur on any particular instance.

Checkpoint — verify before moving on

`tree_csp.py` confirmed a zero-backtrack forward assignment and a matching CP-SAT cross-check, and you explained on paper why an all-`[0, 1, 2]` DAC trace is still a valid witness of Freuder's tree theorem rather than a sign that DAC "did nothing." You are ready to see what happens once the constraint graph is no longer a tree.

3 Unit 2 — Cutset Conditioning, Tree Decomposition and Treewidth

(See Course Notes Week 5, Unit 2 — Cutset Conditioning, Tree Decomposition and Treewidth.)

Step 4: **Run treewidth.py and confirm the tree's induced width is 1**

What will you do? You will run `treewidth.py`, whose `induced_width` function simulates variable elimination on two graphs. First, check its result on Unit 1's own tree graph (R1–R5, the same instance you just ran) against the fact that every tree has treewidth 1.

Why? Running the induced-width calculator on a graph whose answer you already know for certain (a tree, treewidth 1, no exceptions) is what lets you trust the same calculator's answer a moment from now, on a graph you cannot check by eye alone.

The elimination-ordering simulation, applied to the tree graph:

```
31 def induced_width(adjacency: Adjacency, order: list[str]) -> int:
32     """Simulate variable elimination in `order`: at each step, connect the
33     eliminated variable's still-active neighbours into a clique (fill-in
34     edges), and track the largest neighbour count seen at elimination time.
35     """
36     graph = {node: set(neighbours) for node, neighbours in adjacency.items()}
37     width = 0
38     for node in order:
39         neighbours = graph[node]
40         width = max(width, len(neighbours))
41         for neighbour in neighbours:
42             graph[neighbour].update(neighbours - {neighbour})
43         for neighbour in neighbours:
44             graph[neighbour].discard(node)
45         del graph[node]
46     return width
```

```
51 def treewidth_tree_example() -> int:
52     """Unit 1's tree (R1..R5): a tree's treewidth is always 1."""
53     width = induced_width(TREE_GRAPH, TREE_ORDER)
54     print(f"Elimination order: {TREE_ORDER}")
55     print(f"Induced width: {width}")
56     return width
```

Terminal

```
$ python modeller/treewidth.py
```

Expected result: The tree example reports elimination order `['R3', 'R4', 'R5', 'R2', 'R1']` and induced width 1; the cutset example (next step) prints immediately afterwards.

Expected Output

```
Elimination order: ['R3', 'R4', 'R5', 'R2', 'R1']
Induced width: 1
Cutset variable eliminated first: A
Elimination order: ['A', 'B', 'D', 'E', 'C']
Induced width: 3
```

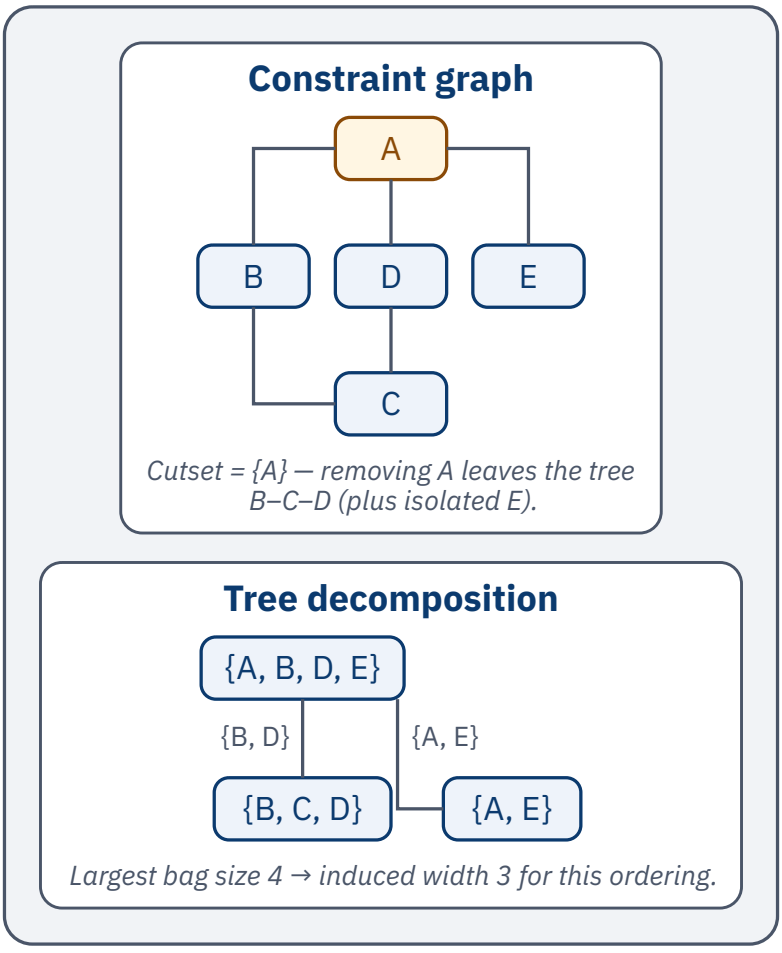
Tip

Both examples run from one `python modeller/treewidth.py` call — the two `ciktikutusu` boxes in this section together are one single terminal output, split here only to match this sheet's two separate steps.

Step 5: Hand-trace the cutset example's induced width

What will you do? The second graph in the output above is a 5-node cycle plus a pendant edge: A–B, B–C, C–D, D–A (the 4-cycle) and A–E. On paper, confirm the printed induced width of `3` by hand: name the cutset variable whose removal leaves a tree, list the fill-in edges created when you eliminate it first, and state the degree that elimination gives it.

Why? This is exactly Unit 2's own worked practice task, on the exact same graph the code just ran on — redoing the arithmetic by hand, then checking it against `treewidth.py`'s own printed answer, is what confirms you can compute an induced width yourself, not just read one off a script's output.



Same graph, two algorithms: condition on the cutset, or decompose into bags.

Expected result: Removing A breaks the 4-cycle A–B–C–D–A into the path B–C–D (E's only edge, A–E, vanishes with A), leaving a tree — so {A} is a cutset. Eliminating A first connects its three remaining neighbours B, D and E into a clique: three fill-in edges, B–D, B–E and D–E, giving A a degree of 3 at elimination time. Eliminating the rest in the order B, D, E, C never exceeds that degree, so the induced width for this ordering is **3** — matching `treewidth.py`'s printed value exactly.

Common Pitfall

It is tempting to count only the cutset variable's *original* degree (3, since A already touches B, D and E before any elimination) and assume that is automatically the induced width. The two numbers agree here, but only because A's neighbours happen to have no fill-in edges added *before* A is eliminated (A is eliminated *first* in this ordering); had a different elimination order been chosen, earlier eliminations could have added fill-in edges among A's neighbours before A's own turn, making A's degree at elimination time larger than its original degree. Always read the width off the graph *at the moment of elimination*, not off the original graph.

Checkpoint — verify before moving on

`treewidth.py` confirmed the tree's induced width of 1 and the cutset example's induced width of 3, and you reproduced the second number by hand, tracing the exact fill-in edges Unit 2's algorithm creates when A is eliminated first. You have now verified both halves of this week's structural claim — a tree costs nothing extra, a small cutset costs a little — on real numbers.

4 Unit 3 — Complexity: What Makes a CSP Hard

(See Course Notes Week 5, Unit 3 — Complexity: What Makes a CSP Hard.)

Step 6: Sketch a constraint graph from a scenario, and classify it

What will you do? A university's timetabling office lists five evening review sessions, V1–V5, for a shared set of overlapping courses. Two students' enrolment records force the following shared-course conflicts: V1–V2, V2–V3, V3–V1 (three sessions overlapping in a triangle) and V3–V4, V4–V5 (a chain hanging off the triangle). On paper: (a) sketch this constraint graph, (b) find its treewidth by trying at least one elimination ordering, and (c) state whether the tree-decomposition algorithm from Unit 2 guarantees a polynomial-time solve for this instance as its variable count grows, and why.

Why? This is Unit 3's own reasoning, applied without any code, to a scenario built specifically so its hard part (the V1–V2–V3 triangle) is small and fixed while its easy part (the V3–V4–V5 chain) could in principle be extended indefinitely — working through both halves yourself is what turns "treewidth explains tractability" from a definition into a habit you can apply to a graph nobody has drawn for you.

Expected result: (a) Five nodes; edges V1–V2, V2–V3, V3–V1 forming a triangle, plus the chain V3–V4, V4–V5 hanging off V3. (b) Eliminating V1 first (degree 2: V2, V3) adds one fill-in edge V2–V3, which already exists — no new edge; eliminating V2 next (now degree 1: V3) removes it with no fill-in; V4 (degree 2: V3, V5) adds fill-in edge V3–V5; the rest are degree-1 removals. The largest degree seen at elimination time is 2, so this ordering's induced width is 2 — and since the graph contains a triangle, treewidth cannot be less than 2 either (a tree, treewidth 1, has no triangle), so the treewidth here is exactly **2**. (c) Yes: this treewidth is a small, *fixed* number (2) that does not grow no matter how many extra chain sessions (V6, V7, ...) are appended to V5 — the tree-decomposition algorithm's polynomial-time guarantee applies here, in sharp contrast to Unit 3's own worked example, where the treewidth grows with the instance.

Checkpoint — verify before moving on

You classified a fresh five-node scenario's tractability entirely on paper — sketching its constraint graph, computing its treewidth by hand, and connecting that single number to Unit 2's polynomial-time guarantee. Unit 4 now turns to a completely different kind of structure: not how tangled a graph is, but how much of it is pure repetition.

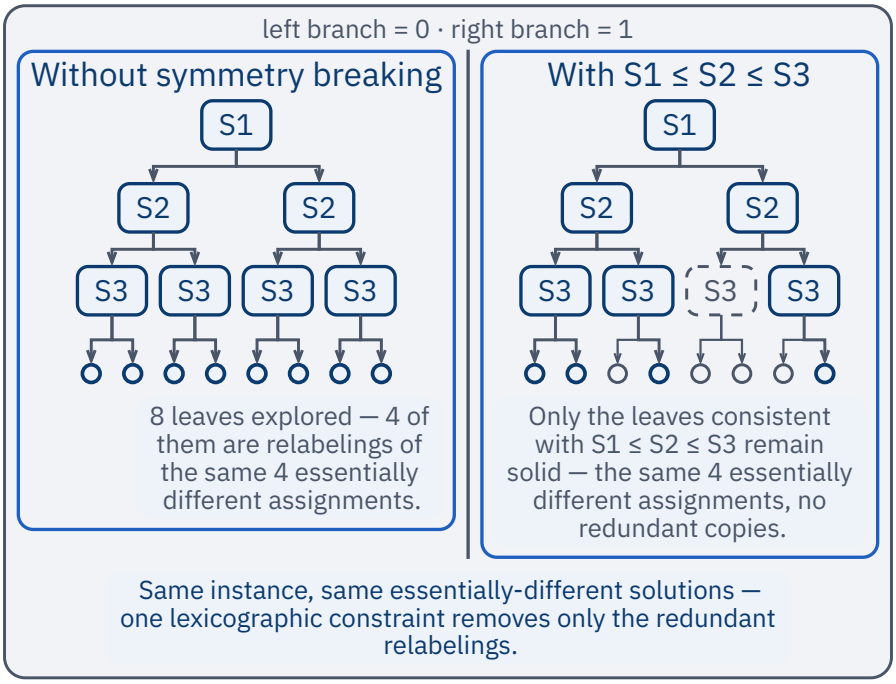
5 Unit 4 — Symmetry and Dominance

(See Course Notes Week 5, Unit 4 — Symmetry and Dominance.)

Step 7: Run social_golfer.py and compare both variants' search statistics

What will you do? You will run `social_golfer.py`, which builds the same 3-groups-of-3-golfers, 4-week instance twice from the same base variables — once with no symmetry-breaking constraints, once with lexicographic-ordering symmetry breaking added — and compare the two runs' `NumBranches`, `NumConflicts` and `WallTime`.

Why? Unit 4 argues, in words, that an unbroken search rediscovers every relabeled copy of the same underlying schedule; this run is your first direct, numeric evidence for that argument, on a real search rather than the small illustrative diagram Unit 4's own figure uses.



The lexicographic-ordering symmetry-breaking constraints:

```
85 def add_symmetry_breaking(model: cp_model.CpModel, play) -> None:
86     """Week 1 fixed to a canonical grouping; groups within every other
87     week ordered by smallest golfer id; weeks after week 1 lexicographically
88     ordered against each other.
89     """
90     for golfer in range(NUM_GOLFERS):
91         model.Add(play[0, golfer, golfer // GROUP_SIZE] == 1)
92     for week in range(1, NUM_WEEKS):
93         model.Add(play[week, 0, 0] == 1)
94     for week in range(1, NUM_WEEKS - 1):
95         left = [_group_of(play, week, golfer) for golfer in range(NUM_GOLFERS)]
96         right = [_group_of(play, week + 1, golfer) for golfer in range(NUM_GOLFERS)]
97         _add_lex_le(model, left, right)
```

Terminal

```
$ python modeller/social_golfer.py
```

Expected result: Both variants report `OPTIMAL`. Symmetry breaking never increases search effort here: branches drop from 2352 to 1454, conflicts drop from 147 to 25, and wall time drops as well (your own machine's exact wall-clock numbers will differ slightly from the values below — compare `NumBranches` and `NumConflicts` instead, since CP-SAT's fixed random seed makes those two deterministic).

Expected Output

```
Without symmetry breaking:
Status: OPTIMAL
NumBranches: 2352
NumConflicts: 147
WallTime: 0.011725
```

Expected Output

```
With symmetry breaking:
Status: OPTIMAL
NumBranches: 1454
NumConflicts: 25
WallTime: 0.006005
```

Tip

The figure above illustrates the same idea on a toy 3-variable instance small enough to draw by hand — 8 leaves shrink to the 4 essentially different assignments once `S1 ≤ S2 ≤ S3` is imposed. The golfer instance is the same effect at a scale too large to draw, visible here only through the branch and conflict counts.

Step 8: Run bibd.py and compare both variants' search statistics

What will you do? You will run `bibd.py`, which models the Fano-plane BIBD $(v, b, r, k, \lambda) = (7, 7, 3, 3, 1)$ twice — once with no symmetry breaking, once with a lexicographic-ordering constraint between every pair of adjacent block-columns — and compare the same three statistics.
Why? This is the same comparison as the previous step, on this week's second combinatorial-design instance; running it on both problems, not just one, is what confirms the branch-and-conflict reduction is a general property of lexicographic symmetry breaking, not a coincidence of the golfer instance's particular numbers.
The hand-rolled sequential lexicographic-$\leq$ helper, since this pinned `ortools` version has no built-in lexicographic constraint:

```python
def add_symmetry_breaking(model: cp_model.CpModel, x) -> None:
    """Adjacent block-columns forced into non-decreasing lexicographic
    order, one column-relabeling symmetry-breaking constraint per pair.
    """
    for block in range(NUM_BLOCKS - 1):
        left = [x[point, block] for point in range(NUM_POINTS)]
        right = [x[point, block + 1] for point in range(NUM_POINTS)]
        _add_lex_le(model, left, right)
```

Terminal

```
$ python modeller/bibd.py
```

Expected result: Both variants report `OPTIMAL` and `NumConflicts` of 0 for both (this instance is small enough that CP-SAT's propagation alone finds a feasible design without ever needing to backtrack), but the branch count still drops from 1185 to 599 once column symmetry breaking is added — fewer regions of the search space are visited even when none of them were ever dead ends.

Expected Output

```
Without symmetry breaking:
Status: OPTIMAL
NumBranches: 1185
NumConflicts: 0
WallTime: 0.005635
```

Expected Output

```
With symmetry breaking:
Status: OPTIMAL
NumBranches: 599
NumConflicts: 0
WallTime: 0.005082
```

Common Pitfall

`NumConflicts: 0` in both variants does not mean symmetry breaking had no effect here — conflicts count backtracks specifically, and this instance is easy enough that neither variant ever hits one. The branch count, which counts every point where the solver commits to a value (including ones that never turn out to be wrong), is the number that still shows symmetry breaking's saving on an instance this small; on a larger, genuinely hard instance you would expect the conflict count to drop as well, the way it did for `social_golfer.py`.

Checkpoint — verify before moving on

Both `social_golfer.py` and `bibd.py` confirmed, on real CP-SAT runs, that lexicographic-ordering symmetry breaking never increases search effort and often reduces it substantially. You have now run every model this week's chapter describes; the final step asks you to extend one of them yourself.

6 Capstone — Add a Second Symmetry-Breaking Constraint

Step 9: **Complete bibd_lab.py's row symmetry breaking and verify it**

What will you do? A starter file, `kod_ornekleri/hafta05/lab/bibd_lab.py`, reuses `bibd.py`'s variables, pairwise-λ constraints and column symmetry breaking unchanged, and adds one more function, `add_row_symmetry_breaking`, marked with a `TODO` that currently just raises `NotImplementedError`. Run the file exactly as given first, to see that error, then complete the function: force every pair of adjacent point-rows into non-decreasing lexicographic order, the same recipe `bibd.py`'s own `add_symmetry_breaking` already applies to adjacent block-columns, reindexed from blocks to points. Re-run the file and confirm it still reports a valid design.

Why? This is this week's integrative step (H5): BIBD's incidence matrix has *two* independent symmetries, point-relabeling (rows) and block-relabeling (columns) — `bibd.py` only ever broke the second one. Adding the first one yourself, on a real running model rather than a written description, is what confirms you can apply Unit 4's lexicographic-ordering recipe to a symmetry nobody has already broken for you.

Terminal

```
$ python lab/bibd_lab.py
```

Expected result: Before your edit: a `NotImplementedError` traceback pointing at `add_row_symmetry_breaking`. After completing the function, a status of `OPTIMAL`, with a branch count of **0** — adding both symmetries at once leaves CP-SAT's presolve able to fix the entire design without any search at all, strictly no worse than column breaking alone (599 branches, previous section).

Expected Output

```
Status: OPTIMAL
NumBranches: 0
NumConflicts: 0
WallTime: 0.002779
```

Tip

The fix is exactly `bibd.py`'s own column-breaking loop, reindexed: iterate `point` from 0 to `NUM_POINTS -- 2`, build `left` and `right` as the row-`point` and row-`(point + 1)` lists of x values across every block, and call `bibd._add_lex_le(model, left, right)` — the same helper that `bibd_lab.py` already imports from `bibd.py`, reused without duplicating a single line of its lexicographic-ordering logic.

Common Pitfall

A common slip is to reuse the *column* loop verbatim instead of reindexing it — iterating over `block` and building `left`/`right` from `x[point, block]` across `point`, which silently just re-adds the column constraint a second time and reports a misleadingly unchanged branch count. Check that your loop variable is `point` (0 to `NUM_POINTS -- 2`) and that `left`/`right` vary over `block` for one *fixed* pair of adjacent points, not the other way around.

Checkpoint — verify before moving on
Your completed `bibd_lab.py` reports a valid Fano-plane design with *both* its point (row) and block (column) symmetries broken at once, and a branch count of 0 — no worse than, in fact better than, the column-only symmetry-broken version. You have now built or verified every piece of this week's chapter, the last one entirely from a written description rather than a fully worked file.

Troubleshooting

Symptom	Likely cause	Fix
`ModuleNotFoundError: No module named 'ortools'`	The virtual environment is not active in this terminal	Re-run `source .venv/bin/activate` (or `Activate.ps1` on Windows) from inside `kod_ornekleri`
Running any `modeller/*.py` or `lab/*.py` script prints `No such file or directory`	The command was run from the wrong working directory	Run `pwd` and confirm you are inside `kod_ornekleri/hafta05` before running any script
`treewidth.py`'s cutset example prints an induced width other than 3	`CUTSET_ORDER` was edited or the elimination started from a node other than A	Restore `CUTSET_ORDER = ["A", "B", "D", "E", "C"]`; Unit 2's worked example specifically eliminates the cutset variable A first
`lab/bibd_lab.py` still raises `NotImplementedError` after you believe you fixed it	The replacement code was written outside the `add_row_symmetry_breaking` function body, or the `raise` line was left in alongside the new code	Open the function again and confirm the `raise NotImplementedError(...)` line itself has been removed, not just commented around
Your completed `bibd_lab.py` reports `INFEASIBLE`	The row loop was built with `point` and `block` swapped, fixing a pair of *blocks* instead of a pair of *points* and duplicating (rather than complementing) the existing column constraint in a way that over-constrains the model	Re-check the `ipucu` box: the loop variable must be `point` (0 to `NUM_POINTS -- 2`), with `left`/`right` varying over `block`
`pytest -q` reports failures unrelated to your own edits	`modeller/*.py` files were edited directly instead of the `lab/` starter file	Restore `modeller/*.py` from the course repository; only `lab/bibd_lab.py` is meant to be edited this week

Transfer — Where Else? 5 · Follow-through

Social golfer and BIBD do not reappear later in this course — both stay in scope only this week — but the two ideas you practiced on them today do. The lexicographic-ordering recipe you just applied a second time, to BIBD's rows, is the exact recipe you will reach for again whenever a later problem's variables come in an interchangeable group: Week 12's timetabling has day-relabeling symmetry, and Week 14's routing has depot-relabeling symmetry, both broken the same way. More generally, the five-minute check you ran on paper in Unit 3 — sketch the constraint graph, ask whether it is close to a tree or hopelessly

tangled, and only then decide whether to expect a fast exact solve — is a habit worth applying to any new combinatorial problem you meet in your own research, before you write a single line of search code. Next week keeps this course's CP-SAT modeling discipline (viewpoints, auxiliary variables, implied constraints, and now symmetry-breaking constraints) but asks a different question: can the exact same model be handed, unchanged, to a completely different solver? You will express one of this week's own symmetry-broken instances in MiniZinc and run it on two solvers with different internals.